Laboratório de Kubernetes

Da Máquina Local à Nuvem Híbrida

Objetivo: Sair do zero, criar um cluster Kubernetes local, testar a auto-recuperação e, por fim, simular uma arquitetura de nuvem híbrida.

INSTRUÇÕES DE ENTREGA DA ATIVIDADE:

1. Execute todos os comandos detalhados neste roteiro prático.
2. Capture telas (*prints*) que comprovem a execução e os resultados dos comandos no seu terminal.
3. Elabore e entregue um **relatório individual** explicando o que foi feito em cada etapa, ilustrado com os seus respectivos *prints*.

ATENÇÃO: Em todos os comandos deste guia, usaremos o prefixo `sudo k3s kubectl`.

Fase 1: O Básico (Comandos e Deploy)

Vamos instalar o cluster e criar a nossa primeira aplicação.

1. Instalar o Kubernetes (k3s)

No seu terminal, execute o comando de instalação do k3s:

```
curl -sfL https://get.k3s.io | sh -
```

Verifique se o nó (sua máquina) está pronto:

```
sudo k3s kubectl get nodes
```

2. Criar um Deployment (Modo Declarativo)

Vamos criar um gerenciador (`Deployment`) que garante 3 réplicas do Nginx:

```
sudo k3s kubectl create deployment meu-app --image=nginx:latest --replicas=3
```

3. Expor e Testar

Exponha a aplicação e veja o serviço criado:

```
sudo k3s kubectl expose deployment meu-app --type=NodePort --port=80
sudo k3s kubectl get services
```

Anote a porta que apareceu no `PORT(S)` (ex: `80:31218/TCP`) e acesse no navegador: `http://localhost:31218`.

Fase 2: Testando Conceito Local (Auto-cura)

Vamos provar que o Deployment mantém o estado desejado.

1. Simule uma falha:
Pegue o nome de um dos pods listados abaixo (será algo como `meu-app-xxxxxxxx-xxxxx`):

```
sudo k3s kubectl get pods
```

Delete o pod (substitua pelo nome que você copiou):

```
sudo k3s kubectl delete pod meu-app-<hash>
```

2. Observando a mágica:
Rode o comando com `-w` (*watch*) para ver em tempo real:

```
sudo k3s kubectl get pods -w
```

> **OBSERVE:** O pod que você deletou entrou em `Terminating` e um **novo pod foi criado automaticamente** em segundos. O Kubernetes percebeu que você queria 3 e recriou o que faltava. *(Aperte `Ctrl+C` para sair do watch).*

3. LIMPEZA (A Transição)

Agora que já entendemos o básico, vamos apagar o `meu-app` para que a nossa próxima aplicação (a distribuída na nuvem) seja a única a rodar o Nginx no cluster. Isso evita confusão na hora de olhar os pods.

```
sudo k3s kubectl delete deployment meu-app
sudo k3s kubectl delete service meu-app
```

(Verifique com `sudo k3s kubectl get pods` se não sobrou nada).

Fase 3: A Nuvem Híbrida (Simulando a AWS)

Agora, vamos colocar uma réplica da sua aplicação no "datacenter do professor".

1. Pegar seus dados de conexão

Pegue o IP da sua máquina e o Token do seu cluster:

```
hostname -I
sudo cat /var/lib/rancher/k3s/server/node-token
```

2. Contratar a Nuvem

Abra o navegador e acesse o portal do professor: `http://<IP_DO_PROFESSOR>:1961`.
Preencha o seu nome, o IP da sua máquina e o Token. Clique em **Conectar**.

3. Verificar a conexão

Rode:

```
sudo k3s kubectl get nodes
```

> **ATENÇÃO:** Agora você verá **dois** nós: o seu computador (*Control Plane*) e um nó com um nome tipo `2945794d5ccd` ou `worker-seu-nome`. **Anote o nome do nó local e o nome do nó da nuvem**, você precisará deles!

4. Deploy Distribuído (O Esqueleto YAML)

Vamos criar uma aplicação com a imagem `nginxdemos/hello`, que mostra o nome do pod que respondeu à requisição.

Crie um arquivo chamado `app-nuvem.yaml` e **preencha os campos em branco**:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-distribuida
spec:
  # PREENCHA AQUI: Quantas replicas queremos? Lembre-se, queremos que existam replicas
     na nuvem e local.
  replicas:
  selector:
    matchLabels:
      app: distribuida
  template:
    metadata:
      labels:
        app: distribuida
    spec:
      # ================================================================================
      # EXERCÍCIO DE ANTI-AFFINITY (O Pulmão na Nuvem)
      # ================================================================================
      # Objetivo: Impedir que o Kubernetes coloque dois Pods desta aplicação
      # na mesma máquina física.
      # ================================================================================
      affinity:
        podAntiAffinity:
          # Usamos "required" (Obrigatório) para forçar a separação. Se a máquina
          # local estiver bloqueada (cordon), o pod ficará "Pending" em vez de
          # desrespeitar a regra.
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  # ----------------------------------------------------------
                  # PREENCHA AQUI: Qual é a chave (key) da label do nosso Pod?
                  # ----------------------------------------------------------
                  # COMO DESCOBRIR:
                  # Rode o comando abaixo no terminal para ver o manifesto
                  # do Deployment que você acabou de criar:
                  #
```

```
                  #   sudo k3s kubectl get deployment app-distribuida -o jsonpath='{.
    spec.selector.matchLabels}'
                  #
                  # O resultado será algo como {"app":"distribuida"}.
                  # A "key" é o que está antes dos dois pontos ("app").
                  - key: [PREENCHA COM A CHAVE DA LABEL]
                    operator: In
                    # ------------------------------------------------------------
                    # PREENCHA AQUI: Qual é o valor (value) dessa label?
                    # ------------------------------------------------------------
                    # COMO DESCOBRIR:
                    # No mesmo resultado do comando anterior, o "value" é
                    # o que está depois dos dois pontos ("distribuida").
                    values: [[PREENCHA COM O VALOR DA LABEL]]

              # ------------------------------------------------------------------
              # PREENCHA AQUI: Qual é a chave que representa uma "máquina física"?
              # ------------------------------------------------------------------
              # COMO DESCOBRIR:
              # 1. Pesquise na documentação do Kubernetes sobre "topologyKey".
              # 2. Ou rode este comando para ler a explicação do próprio K8s:
              #
              #   sudo k3s kubectl explain pod.spec.affinity.podAntiAffinity.
    requiredDuringSchedulingIgnoredDuringExecution.topologyKey
              #
              # O texto dirá que esta chave define o "domínio de topologia".
              # A label padrão para identificar uma máquina física (Nó) é:
              #   kubernetes.io/hostname
              topologyKey: [PREENCHA COM A CHAVE DE TOPOLOGIA]
      containers:
        - name: web
          # PREENCHA AQUI: Use a imagem que mostra o nome do Pod.
          # DICA: Pesquise por "nginxdemos/hello" no Docker Hub (hub.docker.com).
          # A versão que queremos é a "plain-text", que retorna texto simples.
          image:
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: app-distribuida
spec:
  type: NodePort
  selector:
    app: distribuida
  ports:
    - port: 80
      targetPort: 80
      # PREENCHA AQUI: Fixe a porta externa em 30080 para todos acessarem igual
      nodePort:
```

Aplique o arquivo e veja onde os pods caíram:

```
sudo k3s kubectl apply -f app-nuvem.yaml
sudo k3s kubectl get pods -o wide
```

OBSERVE: Na coluna `NODE`, um pod deve estar no **seu nó local** e o outro no **nó da nuvem**. Anote o nome exato do pod que caiu no seu nó local!

5. O Teste de Fogo (Derrubando o Local)

Vamos provar que a nuvem segura a bronca. Vamos simular que o seu datacenter local caiu.

1. Deixe o terminal observando os pods:

```
sudo k3s kubectl get pods -o wide -w
```

2. Em outro terminal, isole o seu nó local: *(Use o nome do nó local anotado)*

```
sudo k3s kubectl cordon <NOME_DO_SEU_NO_LOCAL>
```

3. Simule a queda do pod local: *(Use o nome do pod local anotado)*

```
sudo k3s kubectl delete pod <NOME_DO_POD_LOCAL>
```

O QUE ACONTECEU? Olhe para o terminal do *watch*. O pod que você deletou vai tentar renascer, mas vai ficar `Pending` (Pendente). O Kubernetes está protegendo a sua regra de *Anti-affinity*: ele prefere deixar o pod pendente a colocar dois pods na nuvem.

6. A Prova Final (O Terminal não mente)

Deixe o `curl` rodando em loop para provar que o site não caiu:

```
while true; do curl -s http://localhost:30080 | grep "Server name"; sleep 1; done
```

OBSERVE: O terminal vai mostrar o nome do pod da nuvem respondendo. O site **continua no ar**, servido 100% pelo datacenter do professor! *(Aperte `Ctrl+C` para parar).*

7. Restaurando o Ambiente

Para voltar tudo ao normal e o pod `Pending` voltar para a sua máquina:

```
sudo k3s kubectl uncordon <NOME_DO_SEU_NO_LOCAL>
```

Limpeza Final

Para não sobrecarregar o PC do professor, apague a aplicação da nuvem e destrua o seu cluster local:

```
sudo k3s kubectl delete deployment app-distribuida
sudo k3s kubectl delete service app-distribuida
/usr/local/bin/k3s-uninstall.sh
```